

INTERVIEW GUIDE · BOOK 02

REACT & FULL-STACK

CREATE → UPDATE → OPTIMIZE

The complete React track in three phases, then the Node/Express backend, tools and deployment. Prerequisite: Book 01 — JavaScript. Read in order; each phase assumes the previous one.

06

PARTS

30

TOPICS

12

PAGES

PAIRS WITH BOOK 01 — JAVASCRIPT

2026 EDITION

CONTENTS

React interviews follow the life of a component: you create it, you update it, you optimize it. This book is ordered the same way.

01	CREATE PHASE — SETUP & JSX	Page 03
	npm vs npx · CRA vs Vite · project setup · JSX rules · components · import/export	
02	CREATE PHASE — RENDERING & PROPS	Page 05
	Rendering primitives · lists & keys · ternary · && · ?? · ?. · props · react-router-dom	
03	UPDATE PHASE — EVENTS & HOOKS	Page 07
	Events · useState · useEffect · useRef · useContext · closures in hooks · HOC · Tailwind	
04	OPTIMIZE PHASE	Page 09
	useMemo · useCallback · React.memo · useReducer · useImperativeHandle · Redux Toolkit	
05	BACKEND — NODE & EXPRESS	Page 10
	Node modules · Express · middleware (cors, JWT, bcrypt, multer, dotenv) · HTTP methods · routes & controllers	
06	TOOLS, DEPLOYMENT & PROJECTS	Page 12
	git · Postman & Thunder Client · Vercel · Netlify · Render · GitHub Pages · your 2 project stories	

HOW TO USE THIS BOOK

Build while you read: scaffold one Vite app in Part 1 and grow it through every part — add routing in Part 2, hooks in Part 3, memoization in Part 4, and connect it to your own Express API in Part 5. That app becomes your interview project story.

PROJECT SETUP — NPM, NPX, VITE

NPM VS NPX

npm	Node Package Manager — installs packages into node_modules, records them in package.json (dependencies vs devDependencies). npm install, npm run dev.
npm	Node Package eXecute — runs a package once without installing it globally. Used for scaffolding tools.

SCAFFOLDING — CRA (LEGACY) VS VITE (STANDARD)

```

npx create-react-app myapp      # CRA — webpack, slow, now deprecated
npm create vite@latest myapp    # Vite — pick React → JavaScript
cd myapp
npm install                     # pull dependencies into node_modules
npm run dev                     # dev server → http://localhost:5173

```

WHY VITE WINS

Serves native ES modules in dev — instant start, instant hot reload. Bundles with Rollup only for production. CRA bundles everything up front with webpack on every start.

VITE PROJECT TREE

```

myapp/
├─ index.html      ← root div lives here
├─ package.json
└─ src/
   ├─ main.jsx     ← mounts <App /> into #root
   └─ App.jsx       ← your top component

```

WHERE REACT FITS — AND NEXT.JS

React is a **library for building UI from components**: the UI is a function of state, and React re-renders efficiently via the virtual DOM when state changes. **Next.js** is a framework on top of React adding server-side rendering, file-based routing and API routes — name it, don't claim depth unless you have it.

INTERVIEW Q

"Why Vite over CRA?" — dev speed (ES modules vs full webpack bundle) and CRA's deprecation. "What is the virtual DOM?" — a lightweight copy of the DOM in memory; React diffs old vs new and patches only what changed.

JSX, COMPONENTS, IMPORT/EXPORT

JSX — JAVASCRIPT + XML

JSX compiles to `React.createElement` calls. Four rules: **one parent** (use fragment `<></>`) · **className** not **class** · **expressions in { }** (never if statements) · **Capitalized component names**, and filename = component name.

```
// src/App.jsx — a component is a function that returns JSX
const App = () => {
  const user = "Asha";
  return (
    <>
      <h1>This is React</h1>
      <p sc-camel-class-name="intro">Welcome, {user}</p>    {/* expression in braces */}
    </>
  );
};
export default App;
```

DEFAULT EXPORT VS NAMED EXPORT

DEFAULT — ONE PER FILE

```
export default App;

import App from "./App";
// importer chooses the name
```

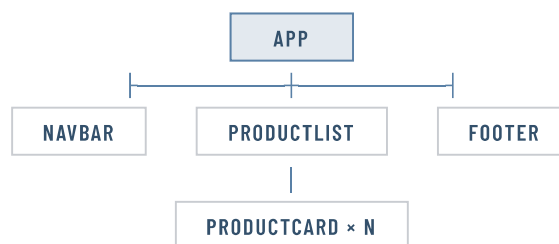
NAMED — MANY PER FILE

```
export const Button = () => {};
export const Input = () => {};

import { Button, Input } from "./ui";
// exact names, in braces
```

FROM SINGLE FILE TO MULTI-FILE COMPONENTS

Start with everything in `App.jsx`; when a block repeats or grows, extract it into its own file and compose. This is the component tree interviewers will ask you to sketch:



INTERVIEW Q

"Why can't you use an if statement inside JSX?" — braces accept only *expressions* (things that produce a value); *if* is a statement. Use a ternary, or run the *if* above the return.

RENDERING METHODS

PRIMITIVE VS NON-PRIMITIVE IN JSX

```
<p>{user.name}</p>      // ✓ string/number render directly
<p>{user}</p>           // ✗ "Objects are not valid as a React child"
{items.map(item => (    // ✓ arrays render item by item → map + key
  <li key={item.id}>{item.title}</li>
))}
```

key tells React which item is which across re-renders — use a stable id, not the array index (index breaks when items reorder or are deleted).

OPTIONAL RENDERING — THE FOUR OPERATORS

OPERATOR	CODE	WHEN
Ternary ? :	{isLoggedIn ? <Dashboard /> : <Login />}	either / or
Logical &&	{error && {error} }	show only if truthy
Nullish ??	{username ?? "Guest"}	right side only when left is null/undefined — a real 0 or "" passes through
Chaining ?.	{user?.address?.city}	undefined instead of a crash on missing data

INTERVIEW Q

"?? vs ||?" — || falls back on ANY falsy value (0 and "" included); ?? only on null/undefined. count ?? 10 keeps a genuine 0. Also: {count && <Badge />} renders a literal 0 when count is 0 — write {count > 0 && ...}.

PROPS — DECLARATION & PRESENTATION

```
// PARENT declares the data going down:
<ProductCard title="Shoes" price={999} sc-camel-on-buy={handleBuy} />

// CHILD receives and presents it (destructured):
const ProductCard = ({ title, price, onBuy }) => (
  <div>
    <h3>{title}</h3><p>₹{price}</p>
    <button sc-camel-on-click={() => onBuy(title)}>Buy</button> // child → parent: call the fn prop
  </div>
);
```

Props flow **one way**, **parent** → **child**, and are **read-only** in the child. Data down, events up — say that sentence in the interview.

REACT-ROUTER-DOM

React apps are SPAs — one HTML page; the router swaps components as the URL changes, without reloading. Install: `npm i react-router-dom`.

```
import { BrowserRouter, Routes, Route, Link,
        useParams, useNavigate, useLocation } from "react-router-dom";

<BrowserRouter>                                {/* wrap once, in main.jsx */}
  <Navbar />                                    {/* outside Routes: on every page */}
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/products" element={<Products />} />
    <Route path="/product/:id" element={<Detail />} />  {/* dynamic segment */}
    <Route path="*" element={<NotFound />} />           {/* 404 */}
  </Routes>
</BrowserRouter>
```

THE FOUR HOOKS + LINK

Link	<code><Link to="/products">Shop</Link></code> — navigation without reload (an <code><a></code> reloads the whole app and loses state).
useParams	<code>const { id } = useParams();</code> — reads <code>:id</code> from <code>/product/42</code> → <code>"42"</code> . Fetch the product with it.
useNavigate	<code>const nav = useNavigate(); nav("/login");</code> — redirect from code, e.g. after logout. <code>nav(-1) = back</code> .
useLocation	<code>const { pathname } = useLocation();</code> — current URL; highlight the active nav item, read passed state.

PATTERN — PROTECTED ROUTE

```
const Protected = ({ children }) =>
  isLoggedIn ? children : <Navigate to="/login" />;

<Route path="/orders" element={<Protected><Orders /></Protected>} />
```

INTERVIEW Q

"Link vs `<a>`?" — `<a>` triggers a full page reload; Link updates the URL and swaps components client-side, preserving state. "How do you read the id from `/product/42`?" — `useParams()`.

EVENTS, USESTATE & USEEFFECT

EVENTS — FUNCTIONS WIRED TO JSX

```
<button sc-camel-on-click={handleSave}>Save</button>           // reference — NO ()
<button sc-camel-on-click={() => removeItem(id)}>Delete</button> // arrow when passing args
<input value={name} sc-camel-on-change={(e) => setName(e.target.value)} /> // controlled input
<form sc-camel-on-submit={(e) => { e.preventDefault(); submit(); }}>
```

USESTATE — STATE THAT RE-RENDERS THE UI

```
const [count, setCount] = useState(0);           // [value, setter]
setCount(count + 1);                             // triggers re-render — async + batched
setCount(prev => prev + 1);                       // updater form: always the fresh value

const [cart, setCart] = useState([]);
setCart([...cart, newItem]);                     // NEVER mutate: new array/object
setCart(cart.filter(i => i.id !== id));           // remove immutably
```

INTERVIEW Q — CLOSER UNDERSTANDING

"Why does console.log show the old state right after setState?" — **closures**: your handler captured the state value from its render; the update lands on the *next* render. Fix stale updates with `setCount(prev => prev + 1)`. This is THE React question — connect it to Book 01's closure page.

USEEFFECT — SIDE EFFECTS WITH A DEPENDENCY ARRAY

```
useEffect(() => {
  fetch("/api/products").then(r => r.json()).then(setProducts);
}, []);           // []           → once, on mount (fetch data here)

useEffect(() => {
  document.title = query;
}, [query]);      // [query]     → every time query changes

useEffect(() => {
  const t = setInterval(tick, 1000);
  return () => clearInterval(t); // CLEANUP — runs on unmount
}, []);           // no array → after EVERY render (rarely wanted)
```

USEREF, USECONTEXT, HOC & TAILWIND

USEREF — DOM ACCESS, NO RE-RENDER

```
const sc-camel-input-ref = useRef(null);
<input ref={inputRef} />
inputRef.current.focus();      // direct DOM access
// Also: a mutable box that survives renders WITHOUT causing them —
// timer ids, previous values. Changing .current re-renders nothing.
```

USECONTEXT — NO MORE PROP DRILLING

```
const AuthCtx = createContext(null);           // 1) create

<AuthCtx.Provider value=>                     // 2) provide (top level)
  <App />
</AuthCtx.Provider>

const { user } = useContext(AuthCtx);          // 3) consume — any depth
```

Prop drilling = passing a prop through components that don't use it, just to reach a deep child. Context solves it for app-wide data: theme, auth, language.

HOC — HIGHER-ORDER COMPONENT

```
const sc-camel-with-auth = (Component) => (props) =>      // takes a component,
  isLoggedIn ? <Component {...props} /> : <Login />; // returns an enhanced one

const SafeDashboard = withAuth(Dashboard);                // same idea as HOFs in JS
```

TAILWIND CSS — UTILITY CLASSES IN JSX

```
<div sc-camel-class-name="flex items-center gap-4 p-4 border">
  <button sc-camel-class-name="px-4 py-2 hover:opacity-80 md:text-lg">Buy</button>
</div>
// know: mobile-first breakpoints (sm: md: lg:), state prefixes (hover: focus:)
```

INTERVIEW Q

"useState vs useRef?" — both persist across renders; useState re-renders on change, useRef doesn't. "When Context vs props?" — props for parent→child data; Context for app-wide data needed at many depths.

MEMOIZATION & USEREDUCER

The rule behind all of these: **a component re-renders when its state or props change — and by default all its children re-render too.** Memoization skips work that would produce the same result.

```
// useMemo — cache an expensive VALUE
const total = useMemo(
  () => items.reduce((sum, i) => sum + i.price, 0),
  [items] // recompute only when items changes
);

// useCallback — cache a FUNCTION reference
const sc-camel-handle-add = useCallback((id) => setCart(c => [...c, id]), []);

// React.memo — child skips re-render when its props are unchanged
const Row = React.memo(({ item, onAdd }) => <li sc-camel-on-click={() => onAdd(item.id)}>{item.name}</li>);
// memo + useCallback work TOGETHER: without useCallback, onAdd is a new
// function every render, so memo's props check always fails.

// useImperativeHandle — expose custom methods through a ref (with forwardRef)
useImperativeHandle(ref, () => ({ focus: () => inputRef.current.focus() }));
```

USEREDUCER — COMPLEX STATE IN ONE PLACE

```
function reducer(state, action) {
  switch (action.type) {
    case "add": return [...state, action.payload];
    case "remove": return state.filter(i => i.id !== action.payload);
    default: return state;
  }
}

const [cart, dispatch] = useReducer(reducer, []);
dispatch({ type: "add", payload: product });
```

Prefer useReducer over useState when the next state depends on the previous one in several ways (cart, form wizard) — it is also the mental model for Redux.

INTERVIEW Q

"useMemo vs useCallback?" — useMemo caches a computed *value*; useCallback caches the *function itself*.
 useCallback(fn, deps) ≡ useMemo(() => fn, deps). Add: "I reach for them only with measurable slowness or memoized children — premature memoization adds complexity."

REDUX TOOLKIT

One global **store** for state many components share (cart, auth). Components never change it directly — they dispatch actions:



```
// store/cartSlice.js
const sc-camel-cart-slice = createSlice({
  name: "cart",
  initialState: [],
  reducers: {                                // "mutating" style is safe here -
    addItem: (state, action) => { state.push(action.payload); }, // Immer
    removeItem: (state, action) => state.filter(i => i.id !== action.payload)
  }
});
export const { addItem, removeItem } = cartSlice.actions;

// store/index.js
export const store = configureStore({ reducer: { cart: cartSlice.reducer } });
// main.jsx: <Provider store={store}><App /></Provider>

// any component:
const items = useSelector(state => state.cart); // READ
const dispatch = useDispatch();
dispatch(addItem(product)); // WRITE
```

INTERVIEW Q

"Context vs Redux?" — Context shares rarely-changing data (theme, auth); Redux adds a predictable update flow, devtools time-travel and middleware for complex, frequently-changing state. "Why Toolkit?" — less boilerplate: createSlice generates actions, Immer allows 'mutating' syntax safely.

NODE, EXPRESS & THE API

Node.js runs JS outside the browser (V8). Three module types: **core** (fs, http, path), **local** (your files), **third-party** (npm). **Express** is the standard web framework on top.

```
const express = require("express");
const app = express();
require("dotenv").config();           // .env secrets → process.env
app.use(require("cors")());           // React :5173 may call API :5000
app.use(express.json());               // parse JSON request bodies

// ROUTE (URL + method) → CONTROLLER (the logic)
app.get("/api/products", getProducts);
app.post("/api/products", auth, createProduct); // auth middleware first

const sc-camel-get-products = (req, res) => res.status(200).json(products);
app.listen(process.env.PORT || 5000);
```

METHOD	USE · STATUS	nodemon	dev: restart server on save
GET	read · 200	bcrypt	hash passwords — hash(pw, 10) / compare(pw, hash); never store plain text
POST	create · 201	JWT	signed login token — sign({id}, SECRET) at login, verify() in middleware; sent as Authorization: Bearer <token>
PUT	replace whole · 200	multer	file uploads (multipart/form-data)
PATCH	update part · 200	dotenv	secrets in .env, out of git
DELETE	remove · 200/204		

THE AUTH FLOW — REHEARSE THIS END TO END



GIT, TESTING & DEPLOYMENT

GIT — THE DAILY LOOP

```
git init                # once per project
git add .               # working dir → STAGING area
git commit -m "add cart" # staging → local history
git push origin main    # local → GitHub (remote)
git checkout -b feature-cart # branch · git merge · git pull
git status · git log    # narrate: working → staging → commit → remote
```

API TESTING · HOSTING

Postman / Thunder Client (VS Code extension): send GET/POST/PUT/DELETE with headers and JSON bodies — test the API before wiring the frontend.

Vercel / Netlify	Frontend hosting — connect the GitHub repo, auto-deploy on every push. Vercel is Next.js-native.
Render	Backend hosting for Node/Express (+ databases) — set env vars in the dashboard.
GitHub Pages	Free static hosting from a repo — portfolios and demos.

YOUR TWO PROJECT STORIES

PROJECT 1 — REACT FRONTEND

Routing, hooks, API fetch, one optimization (memo or debounced search). Deployed on Vercel. This is the app you grew through this book.

PROJECT 2 — FULL-STACK MERN

Express API with JWT auth + bcrypt, CRUD routes, file upload with multer. Backend on Render, frontend on Netlify.

For each, rehearse four beats: the problem · your stack choices and why · one hard bug and how you found it · one optimization with its measurable effect.

NIGHT-BEFORE CHECKLIST

virtual DOM · JSX rules · keys in lists · ?? vs || · props are read-only · useState is async (closures!) · useEffect dependency array · useMemo vs useCallback · Context vs Redux · Link vs <a> · JWT auth flow · GET/POST/PUT/PATCH/DELETE. One breath each — then sleep.